

MACHINE LEARNING, TOLD SIMPLY

Book 5 · Contact Sessions 7 & 8

Twenty Questions

Decision trees — growing a tree with Hunt's algorithm and ID3, measuring impurity with entropy, Gini and classification error, information gain and gain ratio, continuous attributes, and taming overfitting with pre-pruning, post-pruning and the MDL principle.

Machine Learning · AIML ZG565 / DSE ZG565 · BITS Pilani WILP

Companion to Contact Sessions 7 & 8 (Module 5: Decision Tree)

Companion notes prepared by Prof. Saurabh

What is inside

The compliance officer wants a model whose reasons a person can read. This book builds decision trees from the game of Twenty Questions upward: how to score a question (entropy, Gini, error), how to pick the best one (information gain, gain ratio), how to handle numbers, and how to stop a tree from memorising. All ten problem types on the slides are worked in full, including the PlayTennis, EnjoySport, loan-default, customer-ID and shape/colour/size datasets.

1	A Model You Can Read Aloud	What a decision tree is, how it classifies, and why it is the most explainable model in the course.
2	Growing a Tree: Hunt's Algorithm and the Design Questions	The recursive recipe, how a test condition looks for each attribute type, and when to stop.
3	How Mixed Is a Node? Entropy, Gini, Error	Three ways to put a number on impurity — with the intuition of "bits of surprise" behind entropy.
4	Which Question First? Information Gain and ID3	Gain as expected entropy reduction, the full PlayTennis root computation, the second level, and the ID3 algorithm.
5	The EnjoySport Problems, Types 1 to 4	Build a tree, continue on impure nodes, predict a new record, and evaluate with a confusion matrix — with one slide slip corrected.
6	Gini Splits, Continuous Attributes, and Gain Ratio	Problem Types 5 and 6 — binary splits scored with Gini, choosing a threshold for a number, and the fix for the Customer-ID trap.
7	Trees That Memorise: Overfitting and Pruning	Problem Types 7, 8, 9 — comparing tree sizes, pre-pruning with a gain threshold, and post-pruning with a prune set.
8	Occam's Razor With Numbers: The MDL Principle	Problem Type 10 — trading tree size against training errors by counting bits.
9	Formula Sheet, Self-Check, and the Road Ahead	Everything from Sessions 7–8 on one page, twelve questions with answers, and where trees lead next.

How to read this book

Every chapter follows the same rhythm: a short **story** to set the scene, the **big idea** in one line, an everyday **analogy**, then the **maths step by step** — always the plain version first, and the matrix version only after the plain one is clear. Boxes to watch for:

STORY

Meera at Sahi Deal — the running example.

BIG IDEA

The one sentence to remember.

THINK OF IT LIKE

A real-life picture of the concept.

STOP AND THINK

A question to try before reading on.

OUR ANSWER

The answer, right after the question.

CAREFUL

A common mistake and how to avoid it.

MATHS, STEP BY STEP

Every line of algebra, nothing skipped.

WORKED EXAMPLE

Real numbers, worked in full.

EXAM TIP

How this topic is asked, and how to score.

IN ONE BREATH

The chapter in a few lines.

CHAPTER 1

A Model You Can Read Aloud

What a decision tree is, how it classifies, and why it is the most explainable model in the course.

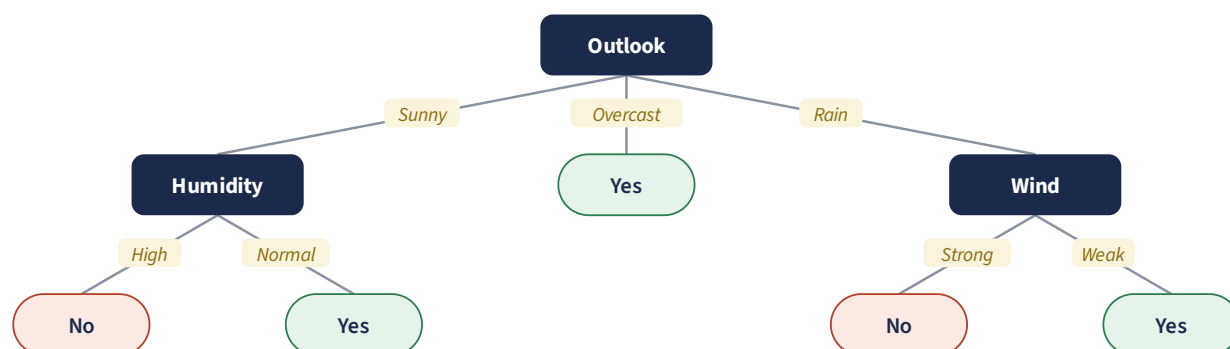
STORY

"Your logistic model says 0.42. What do I tell the customer?" asks the compliance officer. Meera has no sentence to offer — only a weighted sum inside a sigmoid. Arjun sketches on the whiteboard: *Is the applicant a home owner? No. Is the annual income above 80K? No. Then: high risk.* "That," he says, "is a model you can read aloud."

What a decision tree is

BIG IDEA

A decision tree classifies by asking a **sequence of questions about the attributes** of a record. Each answer leads to the next question, until a decision is reached. The questions and answers are arranged as a tree: a **root node** (no incoming edge, two or more outgoing), **internal nodes** (one incoming, two or more outgoing), and **leaf nodes** (one incoming, none outgoing). Every non-leaf node holds a **test condition** on one attribute; every leaf holds a **class label**.



The PlayTennis tree from the slides. Root: *What is the outlook?* Overcast → play. Sunny → ask about humidity. Rain → ask about wind. Leaves are decisions.

THINK OF IT LIKE

The game of Twenty Questions. You want to identify what your friend is thinking of, and every question narrows the possibilities. A good player asks the question that splits the remaining possibilities most evenly ("is it alive?") rather than a wasteful one ("is it a giraffe?"). A decision tree is a Twenty-Questions player that has learnt, from data, which questions to ask first.

Classifying a new record

Start at the root, apply its test to the record, follow the matching branch, repeat until a leaf. The slide's example: $\langle \text{Outlook} = \text{Sunny}, \text{Humidity} = \text{High}, \text{Wind} = \text{Strong} \rangle$. Outlook is Sunny $\rightarrow$ go to the Humidity node; Humidity is High $\rightarrow$ leaf **No**. Wind was never consulted — trees ignore attributes that do not matter on the path taken.

The tree is equivalent to a set of **rules**. Reading each root-to-"Yes" path as an AND, and joining the paths with OR, the PlayTennis tree says:

$$(\text{Outlook} = \text{Sunny} \wedge \text{Humidity} = \text{Normal}) \vee (\text{Outlook} = \text{Overcast}) \vee (\text{Outlook} = \text{Rain} \wedge \text{Wind} = \text{Weak}).$$

This "disjunction of conjunctions" is the tree's **representation**; it is expressive enough for any function of discrete attributes, which is why trees can overfit (Chapter 7).

Classification trees and regression trees

A **classification tree** assigns each leaf a class (fit / unfit; default / no default). A **regression tree** assigns each leaf a number (the average target of the training records that reach it) and so predicts continuous values. This module is about classification trees; the mechanics of regression trees differ only in the impurity measure (variance instead of entropy).

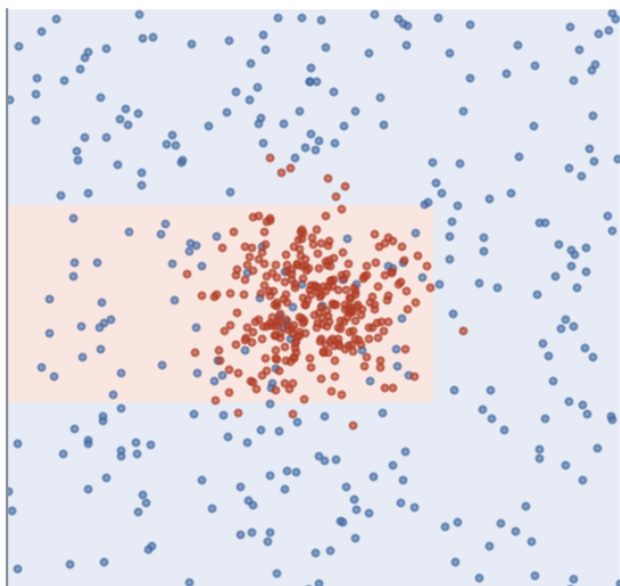
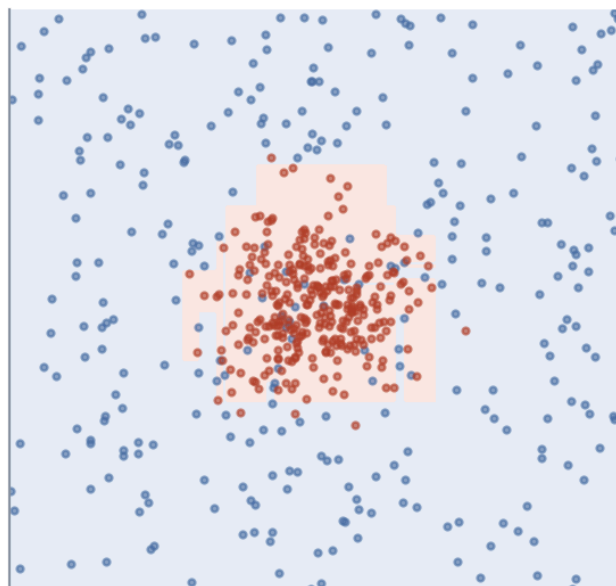
Where trees shine, and where they do not

Advantages (slide)

- ✓ Inexpensive to construct.
- ✓ Extremely fast at classifying unknown records.
- ✓ Easy to interpret for small trees — the compliance officer's wish.
- ✓ Can handle redundant or irrelevant attributes (they simply never get chosen), unless attributes interact.
- ✓ Handle missing values and mixed attribute types; no scaling needed.

Disadvantages (slide)

- The space of possible trees is exponentially large; greedy growth is often unable to find the best tree.
- Greedy splitting does not take attribute interactions into account.
- Each decision boundary involves a single attribute, so boundaries are axis-parallel — a diagonal class boundary needs a staircase of many splits.
- Easy to overfit without pruning.

Tree with 4 leaves: a clean box**Tree with 50 leaves: chasing the noise**

Trees cut the input space into axis-aligned boxes. Left: a small tree captures the blob. Right: a large tree chases individual noisy points.

Where the tree is used

Suited (slide) to problems whose instances are attribute–value pairs, whose target has discrete output values, where disjunctive descriptions may be required, and where training data may contain missing values. Real-world uses: biomedical diagnosis, manufacturing, banking (credit scoring), make-or-buy decisions — anywhere a human must be able to audit the reasoning.

IN ONE BREATH

A tree = questions at the nodes, answers on the branches, decisions at the leaves. Classify by walking root to leaf; explain by reading the path. Fast, interpretable, axis-parallel, and prone to overfitting unless pruned.

CHAPTER 2

Growing a Tree: Hunt's Algorithm and the Design Questions

The recursive recipe, how a test condition looks for each attribute type, and when to stop.

STORY

Meera has ten past loans in front of her. "Which question do I ask first?" "Whichever one sorts them best into repay-ers and defaulters," says Arjun. "Then, inside each group, ask the next best question. Keep going until every group is pure — or until it is silly to continue." That is the whole algorithm; the rest of the book is about the word *best*.

Hunt's algorithm

HUNT'S ALGORITHM (SLIDE)

A decision tree is grown **recursively** by partitioning the training records into **purier subsets**. At a node holding a set of records D_t :

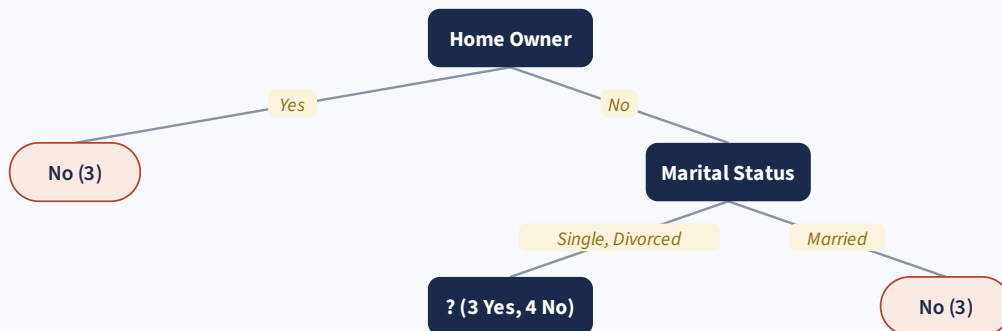
1. **If all records in D_t belong to the same class** → make the node a **leaf** labelled with that class.
2. **If the records belong to more than one class** → choose the best **attribute test condition** to split them; create a child node for each outcome; distribute the records to the children; apply the algorithm to each child.

WORKED EXAMPLE — THE LOAN DATA (SLIDE)

ID	Home Owner	Marital Status	Annual Income	Defaulted?
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Single	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Three attributes of three types — binary, categorical, continuous — and a class. (This is the Session 7–8 version of the table; note ID 6 is Single here.)

Hunt's algorithm as pictured on the slide: (a) the root holds 7 No / 3 Yes — impure, so split. (b) Split on *Home Owner*: the "Yes" child holds IDs 1, 4, 7 — all No — a **leaf**. The "No" child holds 3 Yes / 4 No — split again. (c) Split it on *Marital Status*: Married (IDs 2, 9) are all No — leaf; Single/Divorced (3, 5, 6, 8, 10) are 3 Yes / 2 No — split again. (d) Split on *Annual Income* < 80K: IDs 3 (70K) and 6 (60K) are No; IDs 5, 8, 10 (95K, 85K, 90K) are Yes. Every leaf is now pure.



After two splits. The "?" node still needs the income test to become pure.

Design question 1 — how to express a test condition

The form of a test depends on the attribute type (Book 2's typing pays off here):

Attribute type	Test condition	Example
Binary	Two outcomes	Body temperature: warm-blooded / cold-blooded.
Nominal (many values)	Multi-way split — one branch per value; or binary split by grouping values into two sets ($2^{k-1} - 1$ ways for k values)	Marital status: {Single}, {Married}, {Divorced}; or {Married} vs {Single, Divorced}, etc.
Ordinal	Like nominal, but groupings must respect the order	Shirt size {Small, Medium} vs {Large, XL} is fine; {Small, Large} vs {Medium, XL} is not.
Continuous	Binary : $A < c$ vs $A \geq c$; or multi-way by binning into ranges	Income $> 80K$?; or $\{< 10K, 10-25K, 25-50K, 50-80K, > 80K\}$.

Choosing the threshold c for a continuous attribute is its own procedure — Chapter 5.

Design question 2 — which test is best?

Consider 20 records, 10 of class C_0 and 10 of C_1 , and three candidate tests (slide): *Gender* (Male: 6/4, Female: 4/6), *Car type* (Family: 1/3, Sports: 8/0, Luxury: 1/7), *Customer ID* (20 branches, each with one record). Which is best? Intuitively Gender barely helps — both children are still nearly 50/50. Car type helps a lot — Sports is pure and Luxury nearly so. Customer ID makes every child perfectly pure — but only by memorising, which will be useless on a new customer. We need a **number** that captures "how mixed is a node" (Chapter 3), a rule for combining children (Chapter 4), and a correction for the Customer-ID trap (Chapter 6).

Design question 3 — when to stop splitting

- All records at the node belong to the **same class** → leaf with that class.
- All records have **identical attribute values** (nothing left to split on) → leaf with the **majority class**.
- There are **no records left** for a branch → leaf with the majority class of the parent.
- **Early-stopping constraints** to avoid overfitting: maximum depth, minimum records per node, minimum gain (Chapter 7).

STOP AND THINK

Why can the same attribute not be used twice on one root-to-leaf path in the multi-way-split ID3 algorithm — yet a continuous attribute can appear more than once?

OUR ANSWER

After a multi-way split on a nominal attribute, every record in a child has the *same* value of it, so testing it again separates nothing. A continuous attribute is tested with a threshold ($\text{Income} < 80\text{K}$); the child can still be split at a different threshold ($\text{Income} < 65\text{K}$), so it may reappear deeper down.

IN ONE BREATH

Hunt: if pure, stop; else pick the best test, split, recurse. Tests: binary, multi-way or grouped for nominal/ordinal, threshold for continuous. Stop when pure, when nothing is left to split, or when a size/depth/gain limit says so.

CHAPTER 3

How Mixed Is a Node? Entropy, Gini, Error

Three ways to put a number on impurity — with the intuition of "bits of surprise" behind entropy.

STORY

Meera has two piles of loan files. Pile A: nine repaid, one defaulted. Pile B: five and five. "Which pile would you rather have to guess from?" asks Arjun. "A — I would say 'repaid' and be right 90% of the time." "So A is *purier*. Now give me a number for it."

Entropy

BIG IDEA

Entropy measures the impurity (uncertainty, disorder) of the class distribution at a node. For a node t with C classes, where $p(i | t)$ is the fraction of records at t that belong to class i :

$$\text{Entropy}(t) = - \sum_{i=1}^C p(i | t) \log_2 p(i | t) \quad (\text{with } 0 \log_2 0 := 0).$$

In the binary case, with p_+ and p_- the fractions of positive and negative examples in a set S :

$$\text{Entropy}(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-.$$

THINK OF IT LIKE

Entropy is the number of **yes/no questions you expect to need** to learn the class of a random record from the node — "bits of surprise". If every record is class A, you need zero questions: entropy 0. If the classes are 50/50, you need exactly one question: entropy 1 bit. Anything in between needs a fraction of a question on average. Because $\log_2$ of a number below 1 is negative, the minus sign makes entropy positive; for two classes it lies between 0 and 1, and in general between 0 and $\log_2 C$.

WORKED EXAMPLE — THREE NODES OF SIX RECORDS (SLIDE)

- 1 N_1 : class 0 = 0, class 1 = 6. Entropy = $-\frac{0}{6} \log_2 \frac{0}{6} - \frac{6}{6} \log_2 \frac{6}{6} = 0 - 1 \cdot 0 = 0$. Pure.
- 2 N_2 : 1 and 5. Entropy = $-\frac{1}{6} \log_2 \frac{1}{6} - \frac{5}{6} \log_2 \frac{5}{6} = -\frac{1}{6}(-2.585) - \frac{5}{6}(-0.263) = 0.431 + 0.219 = 0.650$.
- 3 N_3 : 3 and 3. Entropy = $-\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2} = \frac{1}{2} + \frac{1}{2} = 1$. Maximally mixed.

Useful logs: $\log_2 \frac{1}{2} = -1$, $\log_2 \frac{1}{4} = -2$, $\log_2 \frac{1}{3} = -1.585$, $\log_2 \frac{2}{3} = -0.585$, $\log_2 \frac{3}{4} = -0.415$, $\log_2 \frac{1}{5} = -2.322$, $\log_2 \frac{2}{5} = -1.322$, $\log_2 \frac{3}{5} = -0.737$, $\log_2 \frac{4}{5} = -0.322$. And the classics: $H(\frac{1}{2}) = 1$, $H(\frac{1}{3}) = 0.918$, $H(\frac{1}{4}) = 0.811$, $H(\frac{1}{5}) = 0.722$, $H(\frac{5}{14}) = 0.940$.

Gini index

DEFINITION

$$\text{Gini}(t) = 1 - \sum_{i=1}^C [p(i | t)]^2$$

Interpretation: pick two records at random from the node (with replacement); Gini is the probability that they belong to *different* classes. Pure node: 0. Two classes 50/50: $1 - \frac{1}{4} - \frac{1}{4} = 0.5$. Maximum in general: $1 - \frac{1}{C}$. Used by CART (and by scikit-learn's default).

WORKED EXAMPLE (SLIDE)

$N_1(0, 6)$: $1 - 0^2 - 1^2 = 0$. $N_2(1, 5)$: $1 - \left(\frac{1}{6}\right)^2 - \left(\frac{5}{6}\right)^2 = 1 - 0.028 - 0.694 = \mathbf{0.278}$. $N_3(3, 3)$: $1 - \frac{1}{4} - \frac{1}{4} = \mathbf{0.5}$.

Parent node of the loan data, 3 Yes / 7 No: $\text{Gini} = 1 - \left(\frac{3}{10}\right)^2 - \left(\frac{7}{10}\right)^2 = 1 - 0.09 - 0.49 = \mathbf{0.420}$.

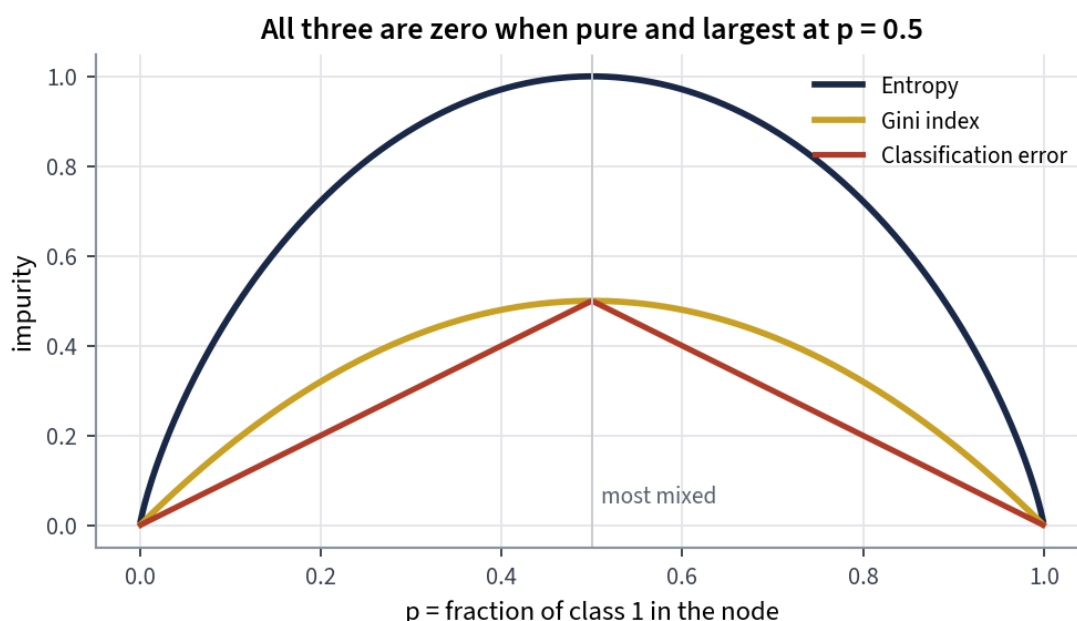
Classification error

DEFINITION

$$\text{Error}(t) = 1 - \max_i p(i | t)$$

$\max_i p(i | t)$ is the fraction you get right by always predicting the majority class; the rest are misclassified. N_1 : $1 - 1 = 0$. N_2 : $1 - \frac{5}{6} = 0.167$. N_3 : $1 - \frac{1}{2} = 0.5$.

The three side by side



All three vanish for a pure node and peak at 50/50. Entropy and Gini are curved (strictly concave); classification error is two straight lines.

Node	Counts (class 0, class 1)	Entropy	Gini	Error
N_1	(0, 6)	0	0	0
N_2	(1, 5)	0.650	0.278	0.167
N_3	(3, 3)	1.000	0.500	0.500

CAREFUL — WHY ERROR IS RARELY USED TO GROW TREES (SLIDE NOTE)

Classification error is *less sensitive to changes in class proportions*: because it is a straight line on each side of $p = 0.5$, a split that moves two children from (4, 4) to (3, 5) and (5, 3) shows *no* improvement in error, while entropy and Gini both register the gain. The curved measures reward every step toward purity; the straight one rewards only steps that change the majority. So Gini or entropy are preferred for growing; error is fine for judging the finished tree.

STOP AND THINK

Three classes, a node with 4, 4 and 4 records. What are its entropy and Gini? Which is now the maximum possible?

OUR ANSWER

Entropy = $-3 \cdot \frac{1}{3} \log_2 \frac{1}{3} = \log_2 3 = 1.585$ bits; Gini = $1 - 3 \cdot \frac{1}{9} = 0.667$. Both are the maxima for $C = 3$ ($\log_2 C$ and $1 - 1/C$). Entropy can exceed 1 as soon as there are more than two classes — the slide's remark "entropy is between 0 and $\log_2 k$ ".

EXAM TIP

Keep a mini log table in your head ($\log_2$ of $\frac{1}{2}, \frac{1}{3}, \frac{1}{4}, \frac{2}{3}, \frac{3}{4}$) and the values 0.918 (1:2 split), 0.811 (1:3), 0.940 (5:9), 0.971 (2:3), 0.722 (1:4). Write the entropy formula once with the fractions substituted, then the decimal; the marker wants to see $-\frac{3}{4} \log_2 \frac{3}{4} - \frac{1}{4} \log_2 \frac{1}{4}$ before 0.811.

IN ONE BREATH

Entropy = $-\sum p \log_2 p$ (bits of surprise; $\max \log_2 C$). Gini = $1 - \sum p^2$ (chance two draws differ; $\max 1 - 1/C$). Error = $1 - \max p$. All are 0 when pure; use entropy or Gini to grow trees.

CHAPTER 4

Which Question First? Information Gain and ID3

Gain as expected entropy reduction, the full PlayTennis root computation, the second level, and the ID3 algorithm.

STORY

Meera can now score a pile. But a question produces *several* piles — one per answer. "Score each child, weight by how many records go there, subtract from the parent," says Arjun. "The question that removes the most uncertainty wins. Do it for every attribute, every time. It is tedious by hand and trivial by computer, but do it once by hand or you will never trust it."

Information gain

BIG IDEA

Information gain is the expected reduction in entropy caused by partitioning the examples on an attribute:

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

S is the set at the node, A the attribute being tested, $\text{Values}(A)$ its possible values, S_v the subset of S with $A = v$. The sum is the **weighted average impurity of the children**. **Maximising the gain = minimising that weighted average**. The higher the gain, the more effective the attribute at classifying the training data.

THINK OF IT LIKE

Twenty Questions again. Before the question you had $H(S)$ bits of uncertainty; after it, on average, $\sum \frac{|S_v|}{|S|} H(S_v)$ bits remain. The gain is the number of bits the question bought you. Ask the question that buys the most.

Worked example — PlayTennis, the root (slides)

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

STEP 1 — ENTROPY OF THE WHOLE SET: 9 YES, 5 NO

$$\text{Entropy}(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = -0.643(-0.637) - 0.357(-1.485) = 0.410 + 0.530 = \mathbf{0.940}$$

STEP 2 — GAIN FOR EACH ATTRIBUTE

Outlook (Sunny 2+/3-, Overcast 4+/0-, Rain 3+/2-):

$$\text{Entropy}(\text{Sunny}) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 0.529 + 0.442 = 0.971, \quad \text{Entropy}(\text{Overcast}) = 0, \quad \text{Entropy}(\text{Rain}) = 0.971$$

$$\text{Gain}(S, \text{Outlook}) = 0.940 - \left[\frac{5}{14}(0.971) + \frac{4}{14}(0) + \frac{5}{14}(0.971) \right] = 0.940 - 0.693 = \mathbf{0.247}$$

Humidity (High 3+/4-, Normal 6+/1-):

$$\text{Entropy}(\text{High}) = -\frac{3}{7} \log_2 \frac{3}{7} - \frac{4}{7} \log_2 \frac{4}{7} = 0.985, \quad \text{Entropy}(\text{Normal}) = -\frac{6}{7} \log_2 \frac{6}{7} - \frac{1}{7} \log_2 \frac{1}{7} = 0.592$$

$$\text{Gain}(S, \text{Humidity}) = 0.940 - \left[\frac{7}{14}(0.985) + \frac{7}{14}(0.592) \right] = 0.940 - 0.789 = \mathbf{0.151}$$

Wind (Weak 6+/2-, Strong 3+/3-):

$$\text{Entropy}(\text{Weak}) = -\frac{6}{8} \log_2 \frac{6}{8} - \frac{2}{8} \log_2 \frac{2}{8} = 0.811, \quad \text{Entropy}(\text{Strong}) = 1.000$$

$$\text{Gain}(S, \text{Wind}) = 0.940 - \left[\frac{8}{14}(0.811) + \frac{6}{14}(1.000) \right] = 0.940 - 0.892 = \mathbf{0.048}$$

Temperature (Hot 2+/2-, Mild 4+/2-, Cool 3+/1-):

$$\text{Gain}(S, \text{Temp}) = 0.940 - \left[\frac{4}{14}(1.000) + \frac{6}{14}(0.918) + \frac{4}{14}(0.811) \right] = 0.940 - 0.911 = \mathbf{0.029}$$

STEP 3 – CHOOSE

$\text{Gain}(\text{Outlook}) = 0.247 > \text{Humidity } 0.151 > \text{Wind } 0.048 > \text{Temperature } 0.029$. **Outlook is the root.** Add a branch for each of its values and partition the 14 days accordingly. The Overcast branch (D3, D7, D12, D13) is all Yes — a leaf immediately.

The second level: the Sunny branch (slide)

WORKED EXAMPLE — $S_{\text{SUNNY}} = \{D1, D2, D8, D9, D11\}$, **2 YES / 3 NO, ENTROPY 0.971**

Humidity: High = {D1, D2, D8} all No; Normal = {D9, D11} all Yes.

$$\text{Gain}(S_{\text{Sunny}}, \text{Humidity}) = 0.971 - \frac{3}{5}(0) - \frac{2}{5}(0) = \mathbf{0.971}$$

Wind: Weak = {D1, D8, D9} (1 Yes, 2 No, entropy 0.918); Strong = {D2, D11} (1, 1, entropy 1).

$$\text{Gain}(S_{\text{Sunny}}, \text{Wind}) = 0.971 - \frac{3}{5}(0.918) - \frac{2}{5}(1.0) = 0.971 - 0.951 = \mathbf{0.020}$$

Temperature: Hot = {D1, D2} (0, 2 → 0); Mild = {D8, D11} (1, 1 → 1); Cool = {D9} (1, 0 → 0).

$$\text{Gain}(S_{\text{Sunny}}, \text{Temp}) = 0.971 - \frac{2}{5}(0) - \frac{2}{5}(1.0) - \frac{1}{5}(0) = \mathbf{0.571}$$

Humidity wins with the maximum possible gain — its children are pure. Both become leaves: High → No, Normal → Yes. (Outlook is not a candidate: every record in this branch is Sunny.)

The Rain branch {D4, D5, D6, D10, D14} (3 Yes, 2 No) works the same way: Wind splits it perfectly (Weak = {D4, D5, D10} all Yes, Strong = {D6, D14} all No), gain 0.971. The finished tree is the one in Chapter 1: leaves {D1, D2, D8} No, {D9, D11} Yes, {D3, D7, D12, D13} Yes, {D4, D5, D10} Yes, {D6, D14} No.

The ID3 algorithm

ID3(X, T, ATTRS) — QUINLAN'S ALGORITHM, AS ON THE SLIDE

X : training examples; T : the target attribute (PlayTennis); Attrs: the other attributes, initially all of them.

1. Create a Root node.
2. If all examples in X are positive, return Root labelled +. If all are negative, return Root labelled −.
3. If Attrs is empty, return Root labelled with the **most common value of T in X** .
4. Otherwise let A = the attribute in Attrs with the **highest information gain**; make A the decision attribute of Root.
5. For each possible value v_i of A : add a branch for $A = v_i$; let X_i = the examples with $A = v_i$. If X_i is empty, add a leaf labelled with the most common value of T in X . Else add the subtree $\text{ID3}(X_i, T, \text{Attrs} - \{A\})$.

6. Return Root.

ID3 is **greedy** (it never revisits a choice), **top-down**, and uses **multi-way splits** on nominal attributes with information gain as the criterion. Its descendants C4.5 (gain ratio, continuous attributes, pruning) and CART (Gini, binary splits, regression trees) are what libraries actually implement.

EXAM TIP

Root-selection questions are long but mechanical. Save time by: (1) computing the parent entropy once; (2) for each attribute, writing the class counts of every child in a tiny table before any logs; (3) reusing known entropies (0.918, 0.811, 0.971, 1); (4) any child that is pure contributes 0 — say so and skip it. Conclude with the gains in a list and the chosen attribute in a box.

IN ONE BREATH

$\text{Gain}(S, A) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v)$. Compute for every attribute, pick the largest, split, recurse on the children with the remaining attributes (ID3). PlayTennis: Outlook (0.247) at the root, Humidity under Sunny, Wind under Rain.

CHAPTER 5

The EnjoySport Problems, Types 1 to 4

Build a tree, continue on impure nodes, predict a new record, and evaluate with a confusion matrix — with one slide slip corrected.

STORY

Before the loan data, Meera practises on the four-row EnjoySport table from Book 2. It is small enough to do everything by hand — and small enough to expose a rounding slip on the slides that students copy every year.

	Sky	AirTemp	Humidity	Wind	Forecast	EnjoySport?
1	Sunny	Warm	Normal	Strong	Same	Yes
2	Sunny	Warm	High	Strong	Same	Yes
3	Rainy	Cold	High	Strong	Change	No
4	Sunny	Warm	High	Strong	Change	Yes

Problem Type 1 — build the tree (which attribute at the root?)

STEP 1 — ENTROPY OF THE CLASS COLUMN: 3 YES, 1 NO

$$E(3, 1) = -\frac{3}{4} \log_2 \frac{3}{4} - \frac{1}{4} \log_2 \frac{1}{4} = -0.75(-0.415) - 0.25(-2) = 0.311 + 0.500 = \mathbf{0.811}$$

CAREFUL — A SLIP ON THE SLIDE

The slide writes $E(3, 1) = 0.4421 + 0.5 = 0.94$. The second term is right (0.5) but the first is not: $-\frac{3}{4} \log_2 \frac{3}{4} = 0.75 \times 0.415 = 0.311$, not 0.442. The value 0.94 belongs to the 9:5 PlayTennis split, not to 3:1. The correct entropy of a 3:1 node is **0.811** (it also appears as "Weak" in PlayTennis). The slide's final choice of root is unaffected, but the gains below are the correct ones — use these in the exam and show the working.

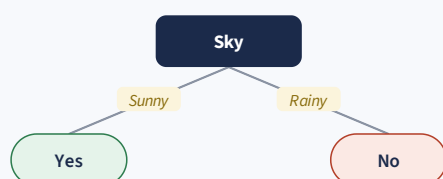
STEP 2 — WEIGHTED CHILD ENTROPY AND GAIN FOR EACH ATTRIBUTE

Attribute	Children (Yes, No)	Weighted entropy of children	Gain = $0.811 - (\cdot)$
Sky	Sunny (3, 0), Rainy (0, 1)	$\frac{3}{4}(0) + \frac{1}{4}(0) = 0$	0.811
AirTemp	Warm (3, 0), Cold (0, 1)	$\frac{3}{4}(0) + \frac{1}{4}(0) = 0$	0.811
Humidity	Normal (1, 0), High (2, 1)	$\frac{1}{4}(0) + \frac{3}{4}\left(-\frac{2}{3}\log_2\frac{2}{3} - \frac{1}{3}\log_2\frac{1}{3}\right) = \frac{3}{4}(0.390 + 0.528) = \frac{3}{4}(0.918) = 0.689$	0.123
Wind	Strong (3, 1) — only one value	$\frac{4}{4}(0.811) = 0.811$	0
Forecast	Same (2, 0), Change (1, 1)	$\frac{2}{4}(0) + \frac{2}{4}(1) = 0.5$	0.311

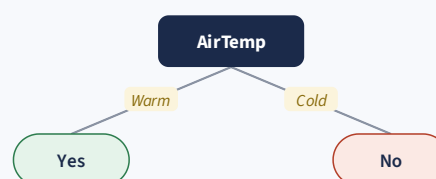
(The slide's Humidity value 0.68 and Forecast value 0.5 agree with these; its Wind value "0.94" should be 0.811, which still gives gain 0.)

STEP 3 — CHOOSE AND DRAW

Sky and AirTemp tie with the maximum possible gain — both separate the data perfectly. Either is a correct root, and each gives a one-level tree with pure leaves:



Tree A: Sky at the root.

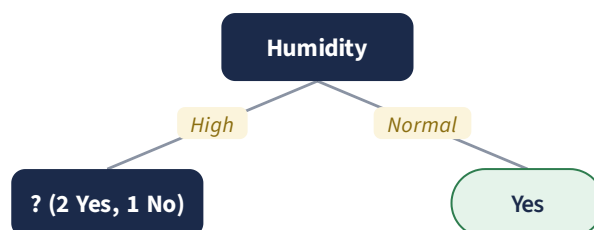


Tree B: AirTemp at the root.

The slide's phrase: *the attribute that generalises the model fastest is the one that reduces randomness most — the one with the highest information gain — Sky and AirTemp*. Both trees are equally correct on the training data.

Problem Type 2 — heterogeneous nodes: keep going

If we had (unwisely) put **Humidity** at the root, the Normal branch would be pure (Yes) but the High branch would hold rows 2, 3, 4 — 2 Yes, 1 No — an impure ("heterogeneous") node.



A root that leaves work undone. The "?" node must be split again.

WORKED EXAMPLE — CONTINUE ON THE HIGH NODE

Records at the node: {2, 3, 4} with attributes Sky (Sunny, Rainy, Sunny), AirTemp (Warm, Cold, Warm), Wind (all Strong), Forecast (Same, Change, Change). Entropy $E(2, 1) = 0.918$.

- Sky: Sunny (2, 0), Rainy (0, 1) → weighted 0 → gain 0.918.
- AirTemp: same partition → gain 0.918.
- Wind: one value → gain 0.
- Forecast: Same (1, 0), Change (1, 1) → $\frac{1}{3}(0) + \frac{2}{3}(1) = 0.667$ → gain 0.251.

Split on Sky (or AirTemp); both children are pure; stop. The slide's **convergence criteria**: repeat for the remaining features until all training data are learnt, no more learning is possible, or a hyper-parameter constraint (depth, minimum samples, minimum gain) is met — *and* the formal stopping rules: all samples at a node share a class; no attributes remain (majority vote labels the leaf); no samples are left.

Problem Type 3 — predict a new instance

WORKED EXAMPLE

New day: ⟨Rainy, Warm, Normal, Breeze, Same⟩. Using Tree A: check Sky → Rainy → right branch → leaf **No**. Predicted EnjoySport = **No**. Notice: Wind = Breeze was never seen in training; because the tree never asks about Wind, that does not matter. (Tree B would ask AirTemp → Warm → **Yes** — the two "equally correct" trees disagree on this new day! With four training rows, both are guessing; this is the inductive leap from Book 2.)

Problem Type 4 — evaluate the model

Evaluation is Book 4's confusion matrix, with two conventions from the slide: by default a model's quality is judged on **unseen validation/test instances**; when a **training** accuracy is wanted for comparison, the training rows are scored too.

WORKED EXAMPLE

Test set: the single new row above, with true label **Yes** (as the slide assumes). Tree A predicts No. Taking Yes as the positive class: $TP = 0$, $FN = 1$, $FP = 0$, $TN = 0$. Test accuracy = $0/1 = 0\%$; recall = 0. Training accuracy: all 4 rows correct, 100%. Interpretation: a perfect training score and a wrong test prediction from a tree built on four examples — the smallest possible illustration of overfitting; with so little data, no tree can be trusted on new days.

The slide's larger practice version (shape/colour/size objects, 9 training rows and 4 test rows) works the same way and gives training accuracy 100%, test accuracy 50%, with the note that **Object ID cannot be used as a splitting feature** — it is an identifier, unique per row, so splitting on it memorises the training set (Chapter 6 shows how gain ratio punishes it).

EXAM TIP

For Type 1, always state the parent entropy with the correct value (0.811 for 3:1) and list every attribute's gain, including the zero for a single-valued attribute. For Type 3, write the path you followed ("Sky = Rainy → No"). For Type 4, define the positive class and the four counts before any metric.

IN ONE BREATH

EnjoySport: $E(3, 1) = 0.811$; Sky and AirTemp both give gain 0.811 and a one-split tree. Impure children get split again with the remaining attributes. Predict by walking the tree; evaluate with a confusion matrix on unseen rows.

CHAPTER 6

Gini Splits, Continuous Attributes, and Gain Ratio

Problem Types 5 and 6 — binary splits scored with Gini, choosing a threshold for a number, and the fix for the Customer-ID trap.

STORY

Meera's loan table has a number in it — annual income — and a column of customer IDs.

"Income is continuous; where do I cut it? And ID gives perfect purity — surely that is the best question?" Arjun: "Cut where the impurity is lowest. And a question that is 'right' only because it has one answer per customer has learnt nothing."

Binary splits scored with Gini (CART style)

WEIGHTED GINI OF A SPLIT

$$\text{Gini}_{\text{split}} = \sum_{j=1}^k \frac{N(v_j)}{N} \text{Gini}(v_j), \quad \text{Gain} = \text{Gini}_{\text{parent}} - \text{Gini}_{\text{split}}$$

Exactly the information-gain structure with Gini in place of entropy. CART restricts itself to **binary** splits, so a nominal attribute with k values is tried in all $2^{k-1} - 1$ two-group forms.

WORKED EXAMPLE — HOME OWNER (SLIDE)

Parent (10 records, 3 Yes / 7 No): $\text{Gini} = 1 - 0.3^2 - 0.7^2 = 1 - 0.09 - 0.49 = 0.420$.

- 1 Child N_1 (Home Owner = Yes: IDs 1, 4, 7): 0 Yes / 3 No. $\text{Gini} = 1 - 0^2 - 1^2 = 0$.
- 2 Child N_2 (Home Owner = No: 7 records): 3 Yes / 4 No. $\text{Gini} = 1 - \left(\frac{3}{7}\right)^2 - \left(\frac{4}{7}\right)^2 = 1 - 0.184 - 0.327 = 0.490$.
- 3 Weighted: $\frac{3}{10}(0) + \frac{7}{10}(0.490) = 0.343$.
- 4 Gain = $0.420 - 0.343 = \mathbf{0.077}$.

WORKED EXAMPLE — MARITAL STATUS, THREE BINARY GROUPINGS (SLIDE)

Single = {1, 3, 6, 8, 10} (3 No, 2 Yes); Married = {2, 4, 9} (3 No); Divorced = {5, 7} (1 Yes, 1 No).

Split	N_1 (No, Yes)	$\text{Gini}(N_1)$	N_2 (No, Yes)	$\text{Gini}(N_2)$	Weighted
{Single} vs {Married, Divorced}	(3, 2)	$1 - 0.36 - 0.16 = 0.48$	(4, 1)	$1 - 0.64 - 0.04 = 0.32$	$\frac{5}{10}(0.48) + \frac{5}{10}(0.32) = 0.400$
{Single, Married} vs {Divorced}	(6, 2)	$1 - \frac{36}{64} - \frac{4}{64} = 0.375$	(1, 1)	0.5	$\frac{8}{10}(0.375) + \frac{2}{10}(0.5) = 0.400$
{Single, Divorced} vs {Married}	(4, 3)	$1 - \frac{16}{49} - \frac{9}{49} = 0.490$	(3, 0)	0	$\frac{7}{10}(0.490) + \frac{3}{10}(0) = 0.343$

The best Marital Status split (0.343) ties with Home Owner (0.343): both isolate a pure group of three non-defaulters. Either is a reasonable root — the slide says "clearly the best candidates, since they both produce the lowest weighted average Gini index".

Problem Type 5 — continuous attributes: where to cut?

Given a continuous attribute A , create a binary attribute $A_c = \text{True}$ if $A < c$. The whole question is the threshold c . Two approaches on the slides:

Way 1 — candidates where the class changes (multi-way flavour)

WORKED EXAMPLE — TEMPERATURE IN PLAYTENNIS

Sort by temperature and read the class underneath: 40 No, 48 No | 60 Yes, 72 Yes, 80 Yes | 90 No. The class changes twice, between 48 and 60 and between 80 and 90. Candidate thresholds are the **midpoints of those pairs**: $(48 + 60)/2 = 54$ and $(80 + 90)/2 = 85$. Evaluate the gain of " $T < 54$ " and " $T < 85$ " (or the three-way split $\{< 54, 54-85, > 85\}$) like any other attribute. Restricting to class-change points loses nothing — a split between two records of the same class can never be optimal.

Way 2 — scan all midpoints and keep the lowest impurity (binary)

WORKED EXAMPLE — ANNUAL INCOME (SLIDE)

- Sort** the incomes with their classes: 60 N, 70 N, 75 N, 85 Y, 90 Y, 95 Y, 100 N, 120 N, 125 N, 220 N.
- Candidate split positions** = midpoints between adjacent values: 65, 72.5, 80, 87.5, 92.5, 97.5, 110, 122.5, 172.5; plus one before the smallest (55) and one after the largest (230).
- For each candidate** count (Yes, No) on each side and compute the weighted Gini. Two of the rows in full:

Split at 55: left is empty ($\text{Gini} = 0$ by convention); right has all 10 (3 Yes, 7 No), $\text{Gini} = 0.42$. Weighted = $\frac{0}{10}(0) + \frac{10}{10}(0.42) = 0.420$ — no better than not splitting.

Split at 97.5: left {60, 70, 75, 85, 90, 95} = 3 Yes, 3 No: $\text{Gini} = 1 - 0.25 - 0.25 = 0.5$. Right {100, 120, 125, 220} = 0 Yes, 4 No: $\text{Gini} = 0$. Weighted = $\frac{6}{10}(0.5) + \frac{4}{10}(0) = 0.300$.

Split $\leq$	55	65	72.5	80	87.5	92.5	97.5	110	122.5	172.5	230
Left (Y, N)	(0,0)	(0,1)	(0,2)	(0,3)	(1,3)	(2,3)	(3,3)	(3,4)	(3,5)	(3,6)	(3,7)
Right (Y, N)	(3,7)	(3,6)	(3,5)	(3,4)	(2,4)	(1,4)	(0,4)	(0,3)	(0,2)	(0,1)	(0,0)
Weighted Gini	0.420	0.400	0.375	0.343	0.417	0.400	0.300	0.343	0.375	0.400	0.420

The minimum is at $c = 97.5$: split "Income $\leq 97.5K$ ". Its weighted Gini 0.300 beats Home Owner and Marital Status (0.343), so in a Gini-based CART tree **income is the root**. (The slide's figure also marks 80 and 110 at 0.343 as the runners-up.)

CAREFUL

Scanning the sorted list is $O(n)$ per attribute once sorted — that is the point of Way 2. And a threshold is a number *between* two data values; quoting " ≤ 95 " instead of " ≤ 97.5 " is the same split on this data but a different rule on new data. Use the midpoint.

Problem Type 6 — the Customer-ID trap and gain ratio

Information gain favours attributes with **many values**: Customer ID has one record per branch, every branch is pure, the gain is the whole parent entropy — and the tree has learnt nothing that transfers to a new customer. (Dates and other identifiers do the same.) C4.5's fix penalises fragmentation:

SPLIT INFORMATION AND GAIN RATIO

$$\text{SplitInformation}(S, A) = - \sum_{i=1}^c \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|}, \quad \text{GainRatio}(S, A) = \frac{\text{Gain}(S, A)}{\text{SplitInformation}(S, A)}$$

SplitInformation is the entropy of the *branch sizes* — how evenly the records are scattered across the c children. Many small children $\Rightarrow$ large SplitInformation $\Rightarrow$ the ratio is pulled down. Two anchors (slide): an ID that splits n records into n singletons has SplitInformation $= -n \cdot \frac{1}{n} \log_2 \frac{1}{n} = \log_2 n$; an attribute that splits into two equal halves has $-2 \cdot \frac{1}{2} \log_2 \frac{1}{2} = 1$.

WORKED EXAMPLE — GENDER, CAR TYPE, CUSTOMER ID (SLIDES)

20 customers, 10 of class C_0 and 10 of C_1 . Parent entropy = 1.

- Gender:** Male 10 (6 C_0 , 4 C_1), Female 10 (4, 6). Each child's entropy $= -0.6 \log_2 0.6 - 0.4 \log_2 0.4 = 0.971$. Children = 0.971; Gain = 0.029. SplitInfo $= -2 \cdot \frac{10}{20} \log_2 \frac{10}{20} = 1$. **GainRatio** = 0.029.
- Car Type:** Family 4 (1, 3): 0.811; Sports 8 (8, 0): 0; Luxury 8 (1, 7): 0.544. Children $= \frac{4}{20}(0.811) + \frac{8}{20}(0) + \frac{8}{20}(0.544) = 0.162 + 0 + 0.218 = 0.380$. Gain = 0.620. SplitInfo $= -\frac{4}{20} \log_2 \frac{4}{20} - 2 \cdot \frac{8}{20} \log_2 \frac{8}{20} = 0.464 + 0.529 + 0.529 = 1.522$. **GainRatio** = 0.620/1.522 = 0.41.
- Customer ID:** 20 children of one record each, all pure: children = 0; Gain = 1 (the maximum). SplitInfo $= \log_2 20 = 4.32$. **GainRatio** = 1/4.32 = 0.23.

Inference (slide): although Customer ID has the highest information gain, its gain ratio (0.23) is lower than Car Type's (0.41) because it produces a larger number of splits. **Choose Car Type.**

THINK OF IT LIKE

Twenty Questions with a cheat: "Is it the word on page 47, line 3?" gets the answer in one question — for that one game. Gain ratio asks how many possible answers the question had; a question with a million possible answers is not clever, it is a lookup.

EXAM TIP

Type 5: sort, list midpoints (including one below the minimum), and show at least two weighted-Gini computations in full plus the table. Type 6: three quantities per attribute — Gain, SplitInformation, and the ratio — then one sentence naming the winner and why the high-gain identifier loses.

IN ONE BREATH

Gini split = $\sum \frac{N_j}{N} \text{Gini}(j)$, lower is better; try all binary groupings. Continuous: sort, midpoints, scan, keep the lowest impurity (income $\leq 97.5\text{K}$, Gini 0.30). GainRatio = Gain / SplitInformation tames many-valued attributes (Car Type 0.41 beats Customer ID 0.23).

CHAPTER 7

Trees That Memorise: Overfitting and Pruning

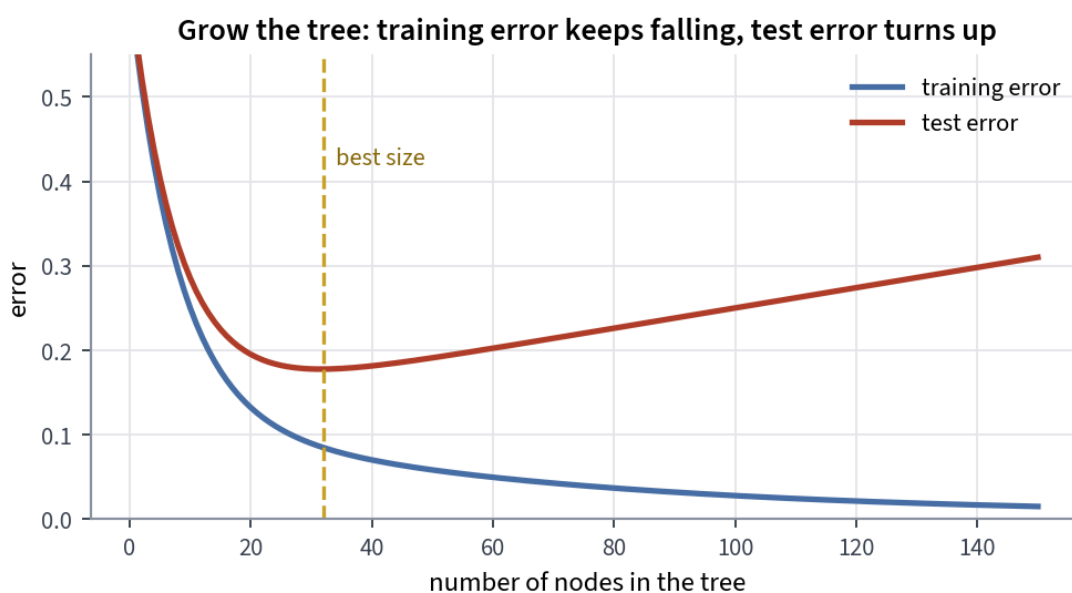
Problem Types 7, 8, 9 — comparing tree sizes, pre-pruning with a gain threshold, and post-pruning with a prune set.

STORY

Meera's full-grown loan tree has 140 nodes and classifies every past loan perfectly. On last month's new loans it is worse than her small hand-drawn tree. "It has one leaf per customer," says Arjun. "Cut it back until it stops knowing the past better than it knows the future."

Problem Type 7 — model overfitting and model comparison

The slide's experiment: two classes, 5,200 instances each — the $+$ class from a Gaussian centred at (10, 10) with 200 noisy points added, the $\circ$ class uniform over the square; 10% used for training, 90% for testing. Growing the tree, **training error keeps falling** toward zero as nodes are added, while **test error falls, bottoms out, then rises**. A four-node tree draws one clean box around the Gaussian; a fifty-node tree carves extra boxes around individual noise points.



Error against tree size. Left of the dashed line: underfitting (both errors high). Right: overfitting (training error tiny, test error growing).

BIG IDEA

Underfitting: the model is too simple; training and test errors are both large. **Overfitting:** the model is too complex; training error is small but test error is large. For trees, "complex" means many nodes.

Which tree is better? — the one with the lower *test* error, never the one with the lower training error.

How noise causes it (slide): add one contradictory day to PlayTennis — ⟨Sunny, Normal, Strong, No⟩. The current tree says Yes for Sunny/Normal, so the algorithm splits that leaf further (on Wind) to fit the new example. The tree now looks "perfect" on training data because it has fitted the noise, and becomes too specific for new data. Overfitting has three named causes: too many branches reflecting **anomalies due to noise or outliers**; **too little data** in deep nodes; and **irrelevant attributes** chosen by chance.

Three remedies

- **Pre-pruning** (early stopping): stop growing when a rule says so.
- **Post-pruning**: grow fully, then cut back with a validation set.
- **Constraints**: a maximum depth, a minimum number of records per leaf — special cases of pre-pruning.

Problem Type 8 — pre-pruning with a gain threshold

Pre-pruning stops the tree early, before it classifies the training data perfectly, using stopping conditions: stop if the information (or Gini) gain is below a threshold; if the node has fewer than a minimum number of samples; if the depth exceeds a maximum; if the improvement in validation accuracy is not significant. The catch: *too strict* a threshold gives a shallow tree that underfits; *too loose* and the tree still overfits.

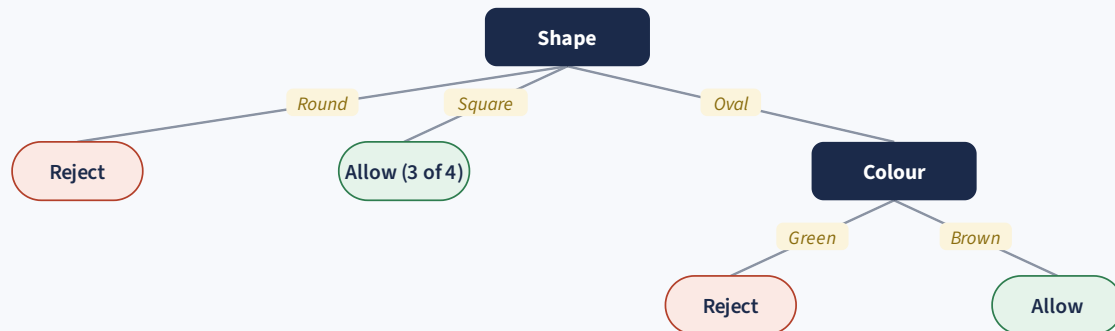
Shape	Colour	Size	Action
Round	Green	Small	Reject
Square	Black	Big	Allow
Square	Brown	Big	Allow
Round	Brown	Small	Reject
Square	Green	Big	Allow
Square	Brown	Small	Reject
Oval	Green	Big	Reject
Oval	Brown	Small	Allow
Oval	Green	Small	Reject

Training set (slide). Test / prune set:
 (Oval, Black, Small, Reject), (Round,
 Brown, Big, Allow), (Square, Brown,
 Big, Allow), (Oval, Green, Small,
 Allow).

WORKED EXAMPLE — THRESHOLD: PRUNE IF GAIN ≤ 0.85

- 1 **Root.** 4 Allow / 5 Reject, entropy 0.991. Shape: Round (0 A, 2 R) $\rightarrow 0$; Square (3 A, 1 R) $\rightarrow 0.811$; Oval (1 A, 2 R) $\rightarrow 0.918$; children = $\frac{2}{9}(0) + \frac{4}{9}(0.811) + \frac{3}{9}(0.918) = 0.667$; gain 0.324. Colour gives 0.186, Size 0.229. **Shape is the root** (the slide starts from here).
- 2 **Round node:** both Reject $\rightarrow$ leaf **Reject**.

- 3 **Square node** (3 Allow, 1 Reject; entropy 0.811). Size: Big (3 A) pure, Small (1 R) pure $\rightarrow$ gain = $0.811 - 0 = \mathbf{0.811}$. Colour: Black (1 A), Brown (1 A, 1 R), Green (1 A) $\rightarrow$ children $\frac{2}{4}(1) = 0.5 \rightarrow$ gain 0.311. Best gain $0.811 \leq 0.85 \rightarrow$ **do not split**; the Square node becomes a leaf with the **majority class Allow** (3 of 4).
- 4 **Oval node** (1 Allow, 2 Reject; entropy 0.918). Colour: Green (2 R), Brown (1 A) $\rightarrow$ pure $\rightarrow$ gain **0.918**. Size: Big (1 R), Small (1 A, 1 R) $\rightarrow$ children $\frac{2}{3}(1) = 0.667 \rightarrow$ gain 0.251. Best gain $0.918 \geq 0.85 \rightarrow$ **split on Colour**: Green $\rightarrow$ Reject, Brown $\rightarrow$ Allow.



The pre-pruned tree. The Square branch was stopped by the threshold and labelled by majority vote.

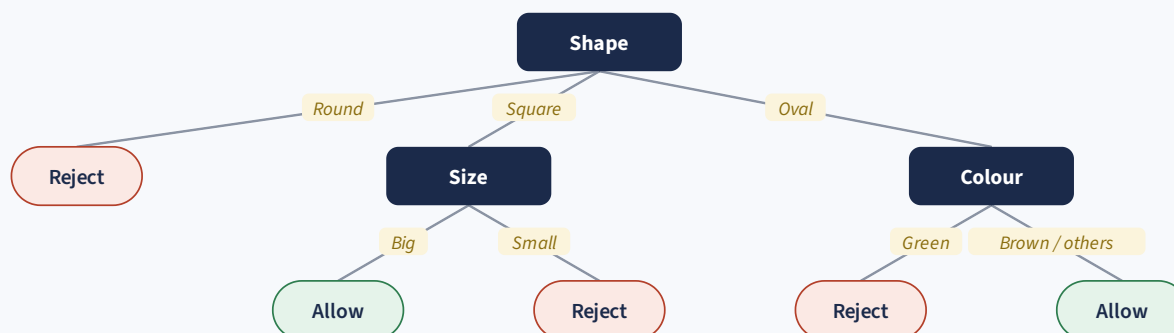
On the test set: (Oval, Black, Small) $\rightarrow$ Oval $\rightarrow$ Colour = Black is unseen — with "Brown/others $\rightarrow$ Allow" the prediction is Allow, but the true label is Reject **X**; (Round, Brown, Big) $\rightarrow$ Reject, true Allow **X**; (Square, Brown, Big) $\rightarrow$ Allow $\checkmark$; (Oval, Green, Small) $\rightarrow$ Reject, true Allow **X**. One of four right — nine training rows simply do not determine these objects. The idea to remember: *while constructing, prune nodes whose best gain is $\leq$ the threshold, and label them by majority vote.*

Problem Type 9 — post-pruning with a prune set

REDUCED-ERROR POST-PRUNING (SLIDE)

1. **Grow the tree fully** — do not stop early; keep splitting until every node is pure (or a minimum leaf size is reached).
2. **Scan the tree bottom-up.**
3. **At every decision node:** keep the node and measure the error on the **prune (validation) set**; then replace the node by a leaf (majority class of its training records) and measure again. If the error **does not increase** (reduces, or stays equal — the simpler tree wins ties), **prune**; otherwise **retain**.
4. Repeat on the other branches.

Error rate is the percentage of prune-set tuples misclassified.

WORKED EXAMPLE (SLIDE) — THE FULLY GROWN TREE AND THE PRUNE SET

Fully grown tree from an intermediate iteration: Square is split on Size, Oval on Colour (Green → Reject; Brown and any other colour → Allow).

Prune-set object	True	Full tree predicts	Prune Size (Square → Allow)	Prune Colour (Oval → Reject)
Oval, Black, Small	Allow	Allow ✓	Allow ✓	Reject ✗
Round, Brown, Big	Reject	Reject ✓	Reject ✓	Reject ✓
Square, Brown, Big	Allow	Allow ✓	Allow ✓	Allow ✓
Oval, Green, Small	Reject	Reject ✓	Reject ✓	Reject ✓
Prune-set error		0 / 4	0 / 4	1 / 4

- 1 Test pruning the Size node.** Replace it by a leaf labelled with the majority of its training records (3 Allow, 1 Reject → Allow). Prune-set predictions are unchanged: error stays 0/4. No increase → **prune** — the tree is simpler for free.
- 2 Test pruning the Colour node.** Majority of its training records (1 Allow, 2 Reject) → Reject. Now (Oval, Black, Small) is called Reject but is truly Allow: error rises to 1/4. Increase → **retain** the Colour split.

Final tree: Shape → Round: Reject; Square: Allow; Oval → Colour (Green: Reject; other: Allow). Same prune-set accuracy, one node fewer.

THINK OF IT LIKE

Editing an essay. Pre-pruning is deciding in advance never to write more than 500 words — safe, but you may stop before the argument is complete. Post-pruning is writing everything, then deleting each paragraph in turn and keeping the deletion whenever a reader (the validation set) understands the essay just as well without it. Post-pruning usually gives the better essay; it just costs more work.

EXAM TIP

Type 8: at each impure node compute the best gain, compare with the threshold, and *write the comparison* (" $0.811 \leq 0.85 \rightarrow \text{leaf, majority Allow}$ "). Type 9: a table with one column per candidate pruning, prune-set error at the bottom, and a decision line ("error unchanged → prune"; "error up → retain"). Always say what label a pruned node gets (majority of its training records).

IN ONE BREATH

More nodes: training error ↓, test error U-shaped. Pre-prune: stop when gain $\leq$ threshold (or depth/size limits), label by majority. Post-prune: grow fully, then bottom-up replace nodes by leaves whenever the prune-set error does not rise.

CHAPTER 8

Occam's Razor With Numbers: The MDL Principle

Problem Type 10 — trading tree size against training errors by counting bits.

STORY

Meera now has two trees: a small one that makes seven mistakes on the training loans and a bigger one that makes four. "Which do I keep?" Arjun: "Imagine you had to email the tree, plus a list of every mistake it makes, to a colleague who has the data but not the labels. The tree that makes the shorter email is the better tree."

The principle

BIG IDEA — MINIMUM DESCRIPTION LENGTH

The best decision tree is the one that **minimises the total description length** of: the tree itself (its structure — nodes, attributes, thresholds) plus the data *encoded using the tree* (the errors or exceptions the tree cannot explain).

$$h_{\text{MDL}} = \arg \min_{h \in H} [L_{C_1}(h) + L_{C_2}(D | h)]$$

$L_{C_1}(h)$ = the number of bits to describe the tree h ; $L_{C_2}(D | h)$ = the number of bits to describe the training labels D given h — which is 0 if h classifies everything perfectly, and otherwise only the **exceptions** need describing.

Occam's razor says prefer the shortest hypothesis. MDL makes "shortest" precise and, crucially, lets a shorter hypothesis with a few errors beat a longer one that is perfect — *trading off hypothesis complexity against the number of errors*. That is exactly the anti-overfitting trade-off in a new costume.

WHERE THE FORMULA COMES FROM (SLIDE)

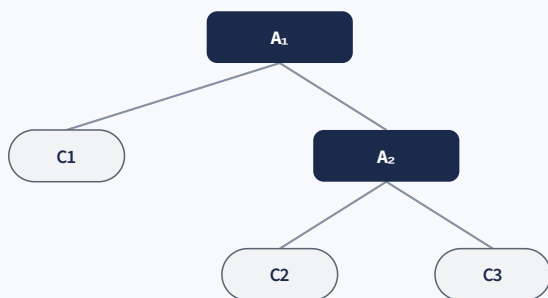
The Bayesian best hypothesis is $h_{\text{MAP}} = \arg \max_h P(D | h)P(h) = \arg \max_h [\log_2 P(D | h) + \log_2 P(h)] = \arg \min_h [-\log_2 P(D | h) - \log_2 P(h)]$. Information theory: the optimal (shortest expected) code for an event of probability p uses $-\log_2 p$ bits. So $-\log_2 P(h)$ is the length of the tree under the best code, $L_{C_1}(h)$, and $-\log_2 P(D | h)$ is the length of the misclassifications, $L_{C_2}(D | h)$. MDL and MAP are the same criterion.

THINK OF IT LIKE

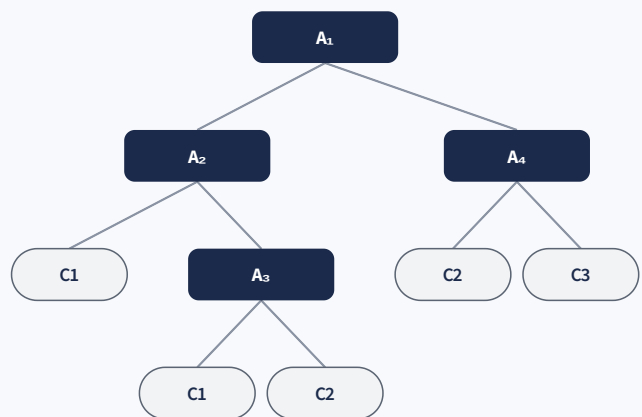
Describing a class photo. "Everyone wore white, except Ravi (blue) and Priya (red)" is short: a simple rule plus two exceptions. Listing all forty shirts one by one is long. Listing a forty-clause rule that predicts every shirt perfectly is also long. The shortest description — rule plus exceptions — is the one that has actually understood the photo.

Problem Type 10 — worked example (slide)**THE QUESTION**

Two decision trees are built from a dataset with **16 binary attributes** and **3 classes** C_1, C_2, C_3 . Tree (a) has 2 internal nodes and 3 leaves and makes **7 errors**; tree (b) has 4 internal nodes and 5 leaves and makes **4 errors**. Compute the total description length of each and decide which is better under MDL.



(a): 2 internal nodes, 3 leaves, 7 errors.



(b): 4 internal nodes, 5 leaves, 4 errors.

THE ANSWER

- 1 **Cost of an internal node.** It must say which of the 16 attributes it tests: $\log_2 16 = 4$ bits.
- 2 **Cost of a leaf.** It must name one of 3 classes: $\lceil \log_2 3 \rceil = 2$ bits.
- 3 **Cost of an error.** It must say *which* of the n training records is the exception: $\log_2 n$ bits each.
- 4 **Tree (a):** $2 \times 4 + 3 \times 2 + 7 \log_2 n = 14 + 7 \log_2 n$ bits.
- 5 **Tree (b):** $4 \times 4 + 5 \times 2 + 4 \log_2 n = 26 + 4 \log_2 n$ bits.
- 6 **Compare.** (a) is better when $14 + 7 \log_2 n < 26 + 4 \log_2 n \iff 3 \log_2 n < 12 \iff \log_2 n < 4 \iff n < 16$. So **tree (a) wins for $n < 16$, tree (b) for $n > 16$** (a tie at $n = 16$).

Read the answer: with few records each error is cheap to list, so the simpler tree wins; with many records an error costs many bits, so it pays to buy a bigger tree that makes fewer of them. MDL automatically becomes more tolerant of complexity as data grows — precisely the behaviour Book 3's learning curves showed.

CAREFUL

The costs are conventions and the question states them (or you state your assumptions). Common ones: an internal node costs $\log_2(\text{number of attributes})$; a leaf costs $\log_2(\text{number of classes})$, rounded up; an error costs $\log_2 n$. Some versions add a bit per node to say "internal or leaf". Write your assumptions, then the arithmetic.

IN ONE BREATH

MDL: minimise (bits for the tree) + (bits for its errors). Node = $\log_2(\# \text{attributes})$, leaf = $\lceil \log_2 \# \text{classes} \rceil$, error = $\log_2 n$. Tree (a) $14 + 7 \log_2 n$ vs (b) $26 + 4 \log_2 n$: (a) better iff $n < 16$.

CHAPTER 9

Formula Sheet, Self-Check, and the Road Ahead

Everything from Sessions 7–8 on one page, twelve questions with answers, and where trees lead next.

The formula sheet

Impurity of a node

Entropy = $-\sum_i p_i \log_2 p_i$ (binary: $-p_+ \log_2 p_+ - p_- \log_2 p_-$; $\max \log_2 C$). Gini = $1 - \sum_i p_i^2$ ($\max 1 - 1/C$). Error = $1 - \max_i p_i$. Handy: $H(1:1) = 1$, $H(1:2) = 0.918$, $H(1:3) = 0.811$, $H(2:3) = 0.971$, $H(1:4) = 0.722$, $H(5:9) = 0.940$.

Choosing a split

Gain(S, A) = $H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v)$; Gini split = $\sum_j \frac{N_j}{N} \text{Gini}(j)$ (lower is better); SplitInfo(S, A) = $-\sum_i \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|}$; GainRatio = Gain/SplitInfo; ID with n singletons: SplitInfo = $\log_2 n$.

Continuous A : sort; candidates = midpoints (where the class changes, or all, plus one below the minimum); pick the lowest weighted impurity.

Growing and stopping

Hunt / ID3: pure $\rightarrow$ leaf; else best attribute, one branch per value, recurse with Attrs $-\{A\}$; empty branch or no attributes left $\rightarrow$ majority class. Stop: pure, identical attributes, no records, or depth / min-samples / min-gain limits.

Overfitting control

Pre-prune: stop if best gain $\leq$ threshold (label by majority). Post-prune (reduced error): grow fully, bottom-up replace a node by its majority leaf if prune-set error does not rise. MDL: minimise $L(\text{tree}) + L(\text{errors})$ with node = $\log_2 \# \text{attr}$, leaf = $\lceil \log_2 \# \text{classes} \rceil$, error = $\log_2 n$.

Twelve questions (cover the right column)

#	Question	Answer
1	Root, internal node, leaf — in-edges and out-edges?	Root: 0 in, ≥ 2 out. Internal: 1 in, ≥ 2 out. Leaf: 1 in, 0 out; holds a class label.
2	Entropy of a node with 1 positive and 5 negatives?	$-\frac{1}{6} \log_2 \frac{1}{6} - \frac{5}{6} \log_2 \frac{5}{6} = 0.431 + 0.219 = 0.650$.
3	Gini of a 3 Yes / 7 No node?	$1 - 0.09 - 0.49 = 0.42$.
4	Why is classification error a poor growth criterion?	It is piecewise linear, so many purity-improving splits show zero gain; entropy and Gini are strictly concave and reward them.
5	Define information gain in words.	The expected reduction in entropy from partitioning on an attribute: parent entropy minus the size-weighted average entropy of the children.
6	PlayTennis root gains?	Outlook 0.247, Humidity 0.151, Wind 0.048, Temperature 0.029 $\rightarrow$ Outlook.
7	Entropy of the EnjoySport class column (3 Yes, 1 No)?	0.811 (not 0.94 — that is the 9:5 value).
8	How are candidate thresholds for a continuous attribute chosen?	Sort the values; take midpoints between adjacent values (at least where the class changes); evaluate each; keep the lowest impurity.
9	Customer ID has the highest gain. Why is it not chosen, and by what measure?	It fragments the data into n singletons (memorises); gain ratio divides by SplitInfo = $\log_2 n$, giving 0.23 vs Car Type's 0.41.
10	Pre-pruning vs post-pruning in one line each.	Pre: stop growing early by a rule (gain threshold, depth, min samples). Post: grow fully, then remove subtrees whose removal does not raise validation error.
11	In reduced-error pruning, what label does a pruned node get?	The majority class of the training records that reached it.
12	MDL: tree (a) costs $14 + 7 \log_2 n$, tree (b) $26 + 4 \log_2 n$. Which is preferred when $n = 64$?	$\log_2 64 = 6$: (a) = 56, (b) = 50 $\rightarrow$ tree (b).

SPARK

A tree is a model that argues in public. Every other model in this course hides its reasoning inside numbers; a tree shows it as a path of questions anyone can follow, challenge, and correct. That transparency is why trees survive in banking, medicine and law — and why, in Module 9, we will let hundreds of them vote (random forests, boosting) to get the accuracy of a black box with the honesty of a tree.

Where the course goes from here

Idea from this book	Where it grows up
Instance-based alternative to a rule	Module 6 — k -nearest neighbours, locally weighted regression, radial basis functions
Linear decision boundaries (Book 4) vs axis-parallel boxes (trees)	Module 7 — support vector machines and the kernel trick
Probabilistic generative classifiers	Module 8 — Bayesian learning, naïve Bayes
Overfitting of a single tree	Module 9 — ensembles: bagging, random forests, AdaBoost, gradient boosting, XGBoost
Learning without labels (Book 1)	Module 10 — k -means, mixture models, EM
Confusion matrices, ROC, bias-variance	Module 11 — comparing models: bias, fairness, interpretability

STORY — THE END OF THE BEGINNING

Meera hands the compliance officer a one-page tree: home owner? income above 97.5K? marital status? Every rejected applicant can be told exactly why, and every rule can be argued with. "It is a little less accurate than the sigmoid," she admits. "Then we will grow a forest of them," says Arjun, "after the mid-term."